

Name: Tejasri M

Reg no: 192511041

Course: Data structure

Code: CSA-0313

Date: 30-7-26

CO2 - AT1 - Analytical problem solving

1) a) operations

→ Append (10)

→ Append (20)

→ Append (15)

Linked list:

After appending (10)

10 → NULL

current = 10

After append (20)

10 → 20 → NULL

current = 20

After append (15)

10 → 20 → 15 → NULL

current = 15

Final list: 10 → 20 → 15 → NULL

Explanation:

The append (.) operation adds each new element at the end of the linked list. Therefore, the final order is 10 → 20 → 15.

b.) Operations

- Append (10)
- Append (20)
- Append (15)
- Move To start()
- Insert (39)
- Next()
- Insert (12)

Explanation:

Move to start() moves the current pointer to the beginning insert() adds a new node before the current position, while next() moves the pointer to the next node.

Linked list:

After Append:

10 → 20 → 15

After Move to start:

current = 10

After Insert (39):

39 → 10 → 20 → 15

After Next():

current = 10

After Insert (12):

39 → 12 → 10 → 20 → 15

Final list : 39 → 12 → 10 → 20 → 15 → NULL

2) Binary Search

Given array:

66 , 111 , 212 , 323 , 432 , 543 , 643 , 649 , 777 , 888

case 1:

key = 888

step	low	High	Mid	value	Result
1	0	9	4	432	Search right
2	5	9	7	649	Search right
3	8	9	8	777	Search right
4	9	9	9	888	Found

Element found in at position 10 (index = 9)

case 2:

key = 111

step	low	high	Mid	Value	Result
1	0	9	4	432	Search left
2	0	3	1	111	Found

Element found in at position 2 (index = 1)

Case 3:

key = 999

Step	low	High	Mid	Value	Result
1	0	9	4	432	Search Right
2	5	9	7	649	Search Right
3	8	9	8	777	Search Right
4	9	9	9	888	Search Right
5	low > high	-	-	-	Not found

Element 999 is not present in the array

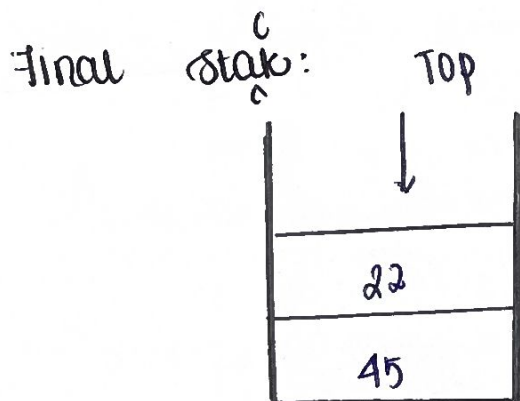
Analysis:

- Binary search works only on sorted array
- It compares the key with the mid element and reduces the search by half
- Time complexity : $O(\log n)$
- Binary search is faster than linear search for large data sets.

3.) Stack Operations:

Stack size = 0

operation	stack (Bottom → Top)
push (45)	45
push (22)	45, 22
push (11)	45, 22, 11, 11
pop()	45, 22, 11
push (44)	45, 22, 11, 44
push (33)	45, 22, 11, 44, 33
pop()	45, 22, 11, 44
pop()	45, 22, 11,
pop()	45, 22



Peak element:

- Top element = 22

- Index position = 1 (0-based indexing)

Time complexity:

push() - $O(1)$

pop() - $O(1)$

Explanation:

The stack follows the LIFO principle. Elements are inserted using push and removed using pop. After performing all the operations, the remaining elements are 45 and 22, with 22 at the top.

A) Convert the expression step by step:

Given:

$((a + b) + c * (d + e) + f) * (g + h)$

a) postfix conversion

Step	scan symbol	Stack	postfix output
1	c	c	
2	(	((	
3	a	((	a
4	+	((+	a
5	b	((+	ab
6	)	(	ab+
7	+	(+	ab+c
8	c	(+c	ab+c
9	*	(+*	ab+c

Step	scan symbol	Stack	postfix output
10	(	(+ * (	ab+c
11	d	(+ * (	ab+cd
12	+	(+ * (+	ab+cd
13	e	(+ * (+	ab+cde
14	)	(+ *	ab+cde+
15	+	(+	ab+cde+*+
16	f	(+	ab+cde+*+f
17	)	NULL	ab+cde+*+f+
18	*	*	ab+cde+*+f+
19	(	* (	ab+cde+*+f+g
20	g	* (*	ab+cde+*+f+g
21	+	* (+	ab+cde+*+f+g
22	h	* (+	ab+cde+*+f+gh
23	)	*	ab+cde+*+f+gh+
24	end	NULL	ab+cde+*+f+gh+

Final postfix - $ab + cde + * + f + gh + *$

b) Prefix conversion

step	operation	Result
1	original expression	$((a+b)+c*(d+e)+f)*(g+h)$
2	Reverse expression	$(h+g)*(f+(e+d)*c+(b+a))$
3	convert to postfix	$+hg+f*+edc+ba+*$
4	Reverse postfix	$*++ab*c+def+gh$

Final prefix: $*++ab*c+def+gh$

Final answers:

prefix - $*++ab*c+def+gh$

postfix - $ab+cde+*+f+gh+*$

5) Priority Queue vs simple Queue

Criteria	Simple Queue	Priority Queue
order of service	Customers are served in the order they arrive (FIFO)	Customers with higher priority are served first
Fairness	Highly, because everyone waits in arrival order	Lower, because normal customers may wait longer

Criteria	Simple Queue	Priority Queue
waiting time	Equal and predictable for all customers	VIP waiting time is reduced, but others may increase.
complexity	Enqueue - $O(1)$ Dequeue - $O(1)$	Insert - $O(\log n)$ Delete - $O(\log n)$

Comparisons:

Simple Queue	Priority Queue
Follows FIFO principle	Follows priority - based service
Fair to all customers	Gives preference to VIP customers
Easy to implement	More efficiently for urgent requests but slightly complex